

TuniTech Abroad

Implementation Roadmap v1

Prepared for MVP implementation planning

1. Document Purpose

This document defines the implementation roadmap for TuniTech Abroad MVP.

It converts the planning documents into a phased build sequence.

It comes after:

- BRD v1
- PRD v1
- Technical Architecture v1
- Database Schema v1
- Service Contracts v1
- Data Flow / Sequence Diagrams v1
- Page + URL + View Map v1

It comes before:

- MVP Backlog v1
- Repository Setup Plan v1
- First Coding Phase

This document answers:

- What do we build first?
- What depends on what?
- What must be tested in each phase?
- What is deferred?
- When is a phase considered done?
- When are we allowed to move to the next phase?

2. Roadmap Principle

The implementation must follow this order:

- Foundation first
- Data model second
- Job data third
- Profile/CV fourth
- Matching fifth
- Recommendations sixth
- LLM support seventh
- Email/privacy/admin polish last

Reason:

- You cannot build matching before jobs and skills exist.
- You cannot build recommendations before matching exists.
- You cannot build weekly digest before recommendations exist.
- You cannot safely use LLM before cache, logging, and limits exist.

3. Hard Rules During Implementation

3.1 No Business Logic in Views

Views should call services.

Bad:

```
view calculates score
view calls OpenRouter
view updates recommendations manually
```

Good:

```
view validates request
view calls service
view renders/redirects
```

3.2 No External API Calls in Models

Forbidden:

```
Model.save() calls France Travail
Model.save() calls OpenRouter
Model method sends email
```

Models store data. Services perform work.

3.3 No Live France Travail Search Per User Query

User search uses local PostgreSQL.

Correct:

```
France Travail API -> ingestion worker -> PostgreSQL -> user search
```

Wrong:

```
User search -> France Travail API
```

3.4 No LLM as Scoring Authority

The deterministic score is final.

LLM can:

```
extract
explain
suggest
summarize
```

LLM cannot:

```
decide final score
change score
bulk-rank all jobs
promise employment
give visa/legal advice
```

3.5 Public URLs Use UUIDs

Public routes must use public_id. Never expose internal integer primary keys.

3.6 CV Soft Delete Must Be Safe

CVUpload.objects must return active/non-deleted CVs only. CVUpload.all_objects is reserved for admin/privacy/deletion tasks.

4. Roadmap Overview

Phase 0: Repository and local environment

Phase 1: Django foundation and authentication
Phase 2: Core database models and admin
Phase 3: Skill taxonomy foundation
Phase 4: Job source, raw jobs, fixtures, normalization
Phase 5: PostgreSQL job search and public job pages
Phase 6: CV upload, parsing, profile completion
Phase 7: Deterministic matching and quick match
Phase 8: Recommendations and saved jobs
Phase 9: OpenRouter LLM support
Phase 10: Email notifications and weekly digest
Phase 11: Privacy deletion and compliance flows
Phase 12: Admin operations and observability
Phase 13: MVP polish and deployment preparation
Phase 14: Production deployment

Phase 0 - Repository and Local Environment

Goal

Create the project workspace and local development foundation.

Deliverables

Git repository
README.md
.env.example
Dockerfile
docker-compose.yml
requirements files
basic folder structure
local PostgreSQL
local Redis
local Django app boot
local Celery worker boot
local Celery Beat boot

Dependencies

None.

Main Tasks

Create repository.
Create Python virtual environment.
Create Django project.
Create apps folder.
Create base/local/production settings split.
Configure PostgreSQL.
Configure Redis.
Configure Celery.
Configure logging.
Create .env.example.
Create basic Makefile or command notes.

Tests

Django starts locally.
PostgreSQL connection works.
Redis connection works.
Celery worker starts.
Celery Beat starts.

Acceptance Criteria

python manage.py runserver works.
python manage.py migrate works.
Celery worker can receive a test task.
PostgreSQL and Redis run through Docker Compose.
README contains local setup instructions.

Stop/Go Gate

Do not continue until local infrastructure is stable.

Phase 1 - Django Foundation and Authentication

Goal

Implement the authentication and base web foundation.

Deliverables

- Custom User model
- django-allauth installed
- email/password signup
- email verification
- Google OAuth placeholders/config
- GitHub OAuth placeholders/config
- login page
- signup page
- logout
- base template
- navigation
- homepage shell
- dashboard access guard

Dependencies

Phase 0 complete.

Main Tasks

- Create accounts app.
- Create custom User model.
- Install/configure django-allauth.
- Configure ACCOUNT_EMAIL_VERIFICATION.
- Configure unique email behavior.
- Configure social login settings.
- Add Google provider.
- Add GitHub provider.
- Create base templates.
- Create homepage.
- Create dashboard placeholder.
- Create post-signup provisioning hook.

Service Work

- AccountProvisioningService
- UserEventService minimal shell

Tests

- User can sign up with email/password.
- Verification email flow works in console/local email backend.
- User can login/logout.
- CandidateProfile is created after signup.
- EmailPreference is created after signup.
- Dashboard requires login.

Acceptance Criteria

- Email/password auth works.
- Google/GitHub settings are prepared.
- AccountProvisioningService is idempotent.
- Authenticated user lands on dashboard.
- Anonymous user cannot access dashboard.

Stop/Go Gate

Do not continue until auth and profile provisioning are stable.

Phase 2 - Core Database Models and Admin

Goal

Create the main database model skeletons and register them in Django Admin.

Deliverables

CandidateProfile
EmailPreference
SystemSetting
ConsentRecord
UserEvent
basic admin registrations
initial migrations

Dependencies

Phase 1 complete.

Main Tasks

Create profiles app.
Create notifications app.
Create privacy app.
Create analytics app.
Create core app.
Add CandidateProfile.
Add EmailPreference.
Add ConsentRecord.
Add UserEvent.
Add SystemSetting.
Register models in admin.
Add base timestamps.
Add indexes where needed.

Service Work

ProfileCompletenessService
SystemSettingService
ConsentService
UserEventService

Tests

Profile completeness calculation works.
SystemSetting fallback works.
Consent can be recorded.
UserEvent can be recorded.
Admin pages load.

Acceptance Criteria

User profile exists for every user.
Profile completeness score can be calculated.
Email preferences exist for every user.
Admin can inspect users/profiles/preferences/events.

Stop/Go Gate

Do not continue until base user/profile/admin structure is stable.

Phase 3 - Skill Taxonomy Foundation

Goal

Build canonical skill taxonomy and alias normalization.

Deliverables

Skill model
SkillAlias model
UnmatchedSkillCandidate model
initial seed command
SkillNormalizerService
admin review support

Dependencies

Phase 2 complete.

Main Tasks

Create skills app.
Create Skill.
Create SkillAlias.
Create UnmatchedSkillCandidate.
Add admin list filters/search.
Build skill normalizer.
Create seed command.
Seed initial 200-300 skills.
Seed initial aliases.

Service Work

SkillNormalizerService
SkillSeedService
UnmatchedSkillReviewService

Tests

Alias maps to canonical skill.
Unknown skill creates UnmatchedSkillCandidate.
Repeated unknown skill increments occurrence count.
Admin can map unknown skill.
Seed command is idempotent.

Acceptance Criteria

Raw skill text can be normalized.
Canonical skills exist.
Unknown skills do not auto-create Skill rows.
Admin review queue works.

Stop/Go Gate

Do not continue until skill normalization is reliable enough for jobs.

Phase 4 - Job Source, Raw Jobs, Fixtures, Normalization

Goal

Build job ingestion foundation without depending immediately on live France Travail availability.

Deliverables

- JobSource
- IngestionRun
- RawJobRecord
- NormalizedJob
- NormalizedJobSkill
- fixture ingestion
- France Travail client skeleton
- job normalization
- job classification
- job freshness logic

Dependencies

Phase 3 complete.

Main Tasks

- Create jobs app.
- Create job source models.
- Create raw/normalized job models.
- Create fixtures for sample France Travail jobs.
- Create fixture ingestion service.
- Create FranceTravailClient skeleton.
- Create JobIngestionService.
- Create JobNormalizationService.
- Create JobClassificationService.
- Create JobFreshnessService.
- Create search_vector field and indexes.

Service Work

- FranceTravailClient skeleton
- JobFixtureIngestionService
- JobIngestionService
- JobNormalizationService
- JobClassificationService
- JobFreshnessService

Celery Tasks

- sync_france_travail_jobs
- normalize_raw_job_record
- mark_stale_and_expired_jobs

Tests

- Fixture raw job creates RawJobRecord.
- RawJobRecord normalizes to NormalizedJob.
- Payload hash prevents unnecessary reprocessing.
- Missing fields do not crash normalizer.
- Job type detection works for stage/PFE/CDI/CDD.
- Stale/expired logic works.
- Search vector gets populated.

Acceptance Criteria

Sample jobs can be ingested from fixtures.
Raw payload is stored.
NormalizedJob is created.
Job status is tracked.
Admin can inspect raw and normalized jobs.
No user-facing search yet required.

Stop/Go Gate

Do not continue until job data can exist locally.

Phase 5 - PostgreSQL Job Search and Public Job Pages

Goal

Expose public job search and job detail pages from local PostgreSQL.

Deliverables

```
/jobs/  
/jobs/<uuid:public_id>/  
JobSearchService  
JobQueryService  
JobRevalidationService shell  
public templates  
pagination  
filters  
job cards  
job detail page
```

Dependencies

Phase 4 complete.

Main Tasks

```
Create public job list view.  
Create job detail view.  
Implement PostgreSQL full-text search.  
Add filters.  
Add pagination.  
Add job card template.  
Add job detail template.  
Add stale/expired labels.  
Add source link display.  
Record job search/detail events.
```

Service Work

```
JobSearchService  
JobQueryService  
JobRevalidationService initial safe shell
```

Tests

```
Anonymous user can search jobs.  
Search uses local DB.  
Inactive/expired jobs hidden from default search.  
Job detail loads by public_id.  
Internal ID route does not exist.  
Filters work.  
Pagination works.
```

Acceptance Criteria

```
Public job search is usable.  
Job details are visible.  
No external API call occurs during normal search.  
public_id routing works.
```

Stop/Go Gate

Do not continue until public job browsing works.

Phase 6 - CV Upload, Parsing, Profile Completion

Goal

Allow authenticated users to upload a CV, parse it, and confirm their profile.

Deliverables

- CVUpload
- CVParsedData
- CVUploadService
- CVTextExtractionService
- CVDeterministicExtractorService
- CVLLMExtractionService shell or disabled mode
- CVParsingService
- CV deletion logic
- profile edit page
- CV management page
- CV parse status partial

Dependencies

Phase 2 complete. Phase 3 complete.

Main Tasks

- Create cvs app.
- Create CVUpload with active/default manager and all_objects.
- Create CVParsedData.
- Add private media handling.
- Add PDF upload form.
- Add CV consent checkbox.
- Add parse_cv task.
- Extract raw text.
- Extract email/phone/links deterministically.
- Add OpenRouter extraction placeholder or feature flag.
- Normalize extracted skills.
- Create ProfileSkill rows.
- Prefill profile draft fields.
- Create profile edit page.
- Create CV delete flow.

Service Work

- CVUploadService
- CVTextExtractionService
- CVDeterministicExtractorService
- CVLLMExtractionService shell
- CVParsingService
- CVDeletionService
- ProfileUpdateService

Celery Tasks

- parse_cv

Tests

- PDF upload accepted.
- Non-PDF rejected.
- File size limit enforced.
- Consent required.

One active CV enforced.
Deleted CV excluded by default manager.
CV text extraction works on sample CV.
Email/phone/links extracted.
ProfileSkill rows created.
Profile can be edited and confirmed.

Acceptance Criteria

Authenticated user can upload CV.
CV parse task runs.
Parsed output is visible.
User can complete profile manually.
Profile completeness updates.
CV can be deleted safely.

Stop/Go Gate

Do not continue to full matching until profile + skills + CV work.

Phase 7 - Deterministic Matching and Quick Match

Goal

Build the core scoring engine and match result pages.

Deliverables

- MatchResult
- QuickMatchSession
- MatchScoringService
- MatchResultService
- QuickMatchService
- job full match endpoint
- anonymous quick match endpoint
- match detail page
- score breakdown UI

Dependencies

Phase 3 complete. Phase 5 complete. Phase 6 complete for full match.

Main Tasks

- Create matching app.
- Create MatchResult.
- Create QuickMatchSession.
- Implement fit score formula.
- Implement technical skill scoring.
- Implement experience scoring.
- Implement role/title scoring.
- Implement language scoring.
- Implement location scoring.
- Implement risk flags.
- Implement profile signals.
- Create full match POST route.
- Create match detail page.
- Create quick match form and HTMX partial.
- Add rate limiting for quick match.

Service Work

- MatchScoringService
- MatchResultService
- QuickMatchService

Tests

- Score formula returns 0-100.
- Required skills affect technical score.
- Missing required skills create risk flag.
- French level missing creates risk flag.
- Profile signals do not reduce score.
- MatchResult stores profile/job snapshots.
- Quick match does not require login.
- Quick match does not call LLM.
- Quick match expires.

Acceptance Criteria

- Authenticated user can generate full match.
- Anonymous user can run quick match.

Score breakdown is explainable.
No LLM needed for score.
Match result is stored.

Stop/Go Gate

Do not continue to recommendations until deterministic matching is stable.

Phase 8 - Recommendations and Saved Jobs

Goal

Generate stored job recommendations using the matching engine.

Deliverables

JobRecommendation
RecommendationRun
SavedJob
RecommendationService
RecommendationQueryService
SavedJobService
dashboard recommendations page
saved jobs page
recommendation stale refresh

Dependencies

Phase 7 complete.

Main Tasks

Create recommendations app.
Create recommendation models.
Create saved jobs model.
Implement RecommendationService.
Implement prefiltering.
Implement ranking score.
Implement stale recommendation behavior.
Implement dashboard recommendation query.
Implement saved/unsaved job buttons.
Implement saved jobs page.
Add recommendation refresh task.

Service Work

RecommendationService
RecommendationQueryService
SavedJobService
ActiveUserRecommendationService

Celery Tasks

refresh_user_recommendations
refresh_active_users_recommendations

Tests

Recommendations generated for usable profile.
Incomplete profile skips recommendations.
Ranking score uses fit score + boosts.
Duplicate recommendations are not created.
Profile update marks recommendations stale.
Dashboard does not block on refresh.
Saved job is idempotent.
Expired saved jobs show status.

Acceptance Criteria

User sees recommended jobs.
Recommendations are precomputed.
Dashboard reads stored recommendations.
Saved jobs work.
Recommendations refresh after profile/CV changes.

Stop/Go Gate

Do not continue to email digest until recommendations work.

Phase 9 - OpenRouter LLM Support

Goal

Add controlled LLM support for structured CV extraction and match explanations.

Deliverables

PromptVersion
LLMCacheEntry
LLMUsageLog
OpenRouterClient
PromptRunnerService
LLMRateLimitService
LLMOutputValidationService
CV structured extraction enabled
match explanation endpoint

Dependencies

Phase 6 complete. Phase 7 complete.

Main Tasks

Create llm app.
Create prompt version models.
Create cache model.
Create usage log model.
Implement OpenRouter client.
Implement PromptRunnerService.
Implement cache keys.
Implement rate limits.
Implement output validation.
Enable CV LLM extraction.
Enable match explanation.
Add admin monitoring.

Service Work

OpenRouterClient
PromptRunnerService
LLMRateLimitService
LLMOutputValidationService
CVLLMExtractionService
MatchExplanationService

Tests

Prompt version resolves correctly.
Cache hit avoids API call.
Rate limit blocks excess calls.
LLM failure does not break match.
LLM explanation cannot change score.
CV extraction output is validated.
Usage logs are recorded.

Acceptance Criteria

OpenRouter calls are isolated.
LLM output is cached.
LLM usage is logged.
Daily limits work.

Match explanations work.
Core app works if LLM is disabled/fails.

Stop/Go Gate

Do not continue to production until LLM cost controls work.

Phase 10 - Email Notifications and Weekly Digest

Goal

Add transactional emails and opt-in weekly recommendation digest.

Deliverables

- EmailBatch
- EmailEvent
- EmailUnsubscribeToken
- EmailSenderService
- EmailPreferenceService
- WeeklyDigestService
- email preference page
- unsubscribe link
- weekly digest task

Dependencies

Phase 8 complete. Phase 1 email setup complete.

Main Tasks

- Create notifications app if not already done.
- Implement email provider abstraction.
- Implement idempotent EmailEvent.
- Implement EmailBatch.
- Implement email preferences page.
- Implement unsubscribe token flow.
- Implement weekly digest generation.
- Implement digest templates.

Service Work

- EmailSenderService
- EmailPreferenceService
- WeeklyDigestService

Celery Tasks

- send_weekly_digest

Tests

- Email idempotency prevents duplicates.
- Weekly digest only sends to opted-in users.
- Unverified email users excluded.
- Inactive users excluded.
- Unsubscribe disables digest.
- EmailEvent logs status.

Acceptance Criteria

- Password reset/verification email works.
- Weekly digest can be enabled/disabled.
- Weekly digest is idempotent.
- Unsubscribe works without login.

Stop/Go Gate

Do not enable scheduled digest until idempotency test passes.

Phase 11 - Privacy Deletion and Compliance Flows

Goal

Implement privacy-safe deletion and consent handling.

Deliverables

Privacy page
Terms page
Consent records
Delete CV
Delete account
DeletionRequest
AccountDeletionService
privacy deletion task

Dependencies

Phase 6 complete. Phase 8 complete.

Main Tasks

Create privacy/terms pages.
Record CV processing consent.
Record email digest consent.
Implement delete CV flow.
Implement delete account page.
Implement deletion request.
Implement deletion worker.
Anonymize/delete UserEvent data.
Ensure CV physical files are deleted.

Service Work

ConsentService
AccountDeletionService
CVDeletionService hardened

Celery Tasks

process_account_deletion
cleanup_expired_quick_matches
delete_orphaned_cv_files

Tests

CV deletion removes file.
Deleted CV is not returned by default manager.
Account deletion disables login.
Account deletion removes CV files.
Account deletion removes profile/matches/recommendations.
DeletionRequest completed.
Failed deletion can be retried.

Acceptance Criteria

User can delete CV.
User can request account deletion.
Private candidate data is removed/anonymized.

Consent is recorded.
Privacy/terms pages exist.

Stop/Go Gate

Do not deploy publicly without privacy deletion working.

Phase 12 - Admin Operations and Observability

Goal

Make the system operable through Django Admin and health checks.

Deliverables

- admin list filters/search
- admin actions
- health endpoint
- ingestion monitoring
- LLM usage monitoring
- email monitoring
- unknown skill review
- basic event analytics

Dependencies

All core model phases complete.

Main Tasks

- Register all models in admin.
- Add useful `list_display`.
- Add `search_fields`.
- Add `list_filter`.
- Add admin actions for job reprocessing.
- Add unmatched skill admin review.
- Add recommendation refresh admin action.
- Add CV reparse admin action.
- Add health endpoint.
- Add dashboard/admin visibility for failed jobs.

Tests

- Staff can access admin.
- Non-staff cannot access admin.
- Admin actions call services.
- Health endpoint checks DB/Redis.
- Failed ingestion visible.
- Failed CV parse visible.

Acceptance Criteria

- Admin can operate MVP without shell access for common tasks.
- Health endpoint works.
- Unknown skills are reviewable.
- LLM and email logs are visible.

Stop/Go Gate

Do not deploy if admin cannot inspect failures.

Phase 13 - MVP Polish and Deployment Preparation

Goal

Prepare MVP for first controlled external use.

Deliverables

- error pages
- empty states
- loading states
- form validation polish
- responsive layout
- French UI text pass
- security settings
- production settings
- deployment checklist
- seed commands
- fixture commands
- test data

Dependencies

Phases 0-12 complete.

Main Tasks

- Polish public pages.
- Polish dashboard.
- Add empty states.
- Add error states.
- Add 404/500 pages.
- Review French wording.
- Review privacy wording.
- Add production settings.
- Add static file handling.
- Add media private storage rules.
- Add seed management commands.
- Add deployment documentation.

Tests

- Main flows manually pass.
- Forms show useful errors.
- Responsive layout works.
- Security settings are production-ready.
- Static files work.
- Private media not publicly accessible.

Acceptance Criteria

- MVP is usable by first test users.
- No broken critical page.
- No exposed CV files.
- No public integer IDs.
- French UI consistent enough.

Stop/Go Gate

Do not deploy until critical manual flows pass.

Phase 14 - Production Deployment

Goal

Deploy the MVP for controlled validation.

Deliverables

production database
production Redis
web service
worker service
beat service
domain
HTTPS
environment variables
admin account
initial seed data
scheduled ingestion enabled
monitoring checklist
backup plan

Dependencies

Phase 13 complete.

Main Tasks

Choose deployment platform.
Configure environment variables.
Run migrations.
Seed system settings.
Seed skills.
Seed France Travail source.
Create admin.
Configure OAuth redirect URLs.
Configure email provider.
Run fixture ingestion.
Run real ingestion if credentials ready.
Start web/worker/beat.
Verify health endpoint.

Tests

Production /health/ works.
Login works.
Admin works.
Job search works.
CV upload works.
Celery worker processes tasks.
Email sends.
Private media protected.

Acceptance Criteria

MVP is live.
Admin can log in.
Jobs can be searched.
CV upload works.
Matching works.
Recommendations work.
No critical security issue.

5. Deferred Features

Do not build before MVP validation:

- Belgium jobs
- Luxembourg jobs
- Canada jobs
- Switzerland jobs
- La Bonne Alternance integration
- Recruiter dashboard
- Training center dashboard
- Payments
- Paid reports
- Auto-apply
- Chatbot
- Salary prediction
- Company reviews
- LinkedIn login
- Facebook login
- Full LinkedIn profile import
- OCR for scanned CVs
- Meiliseach
- Mobile app
- Public candidate marketplace

6. Critical Path

The shortest path to useful MVP is:

- Phase 0
- > Phase 1
- > Phase 2
- > Phase 3
- > Phase 4
- > Phase 5
- > Phase 6
- > Phase 7
- > Phase 8

At the end of Phase 8, the product has the core loop:

- jobs
- CV/profile
- matching
- recommendations
- saved jobs

Phases 9-14 improve intelligence, communication, privacy hardening, operations, and deployment.

7. Build Order Warning

Do not start with:

- LLM
- emails
- beautiful UI
- deployment
- payments
- multi-country support

Start with:

- models
- auth
- skills
- jobs
- CV/profile

matching
recommendations

The product value is the data loop, not visual polish.

8. Testing Expectations by Layer

8.1 Unit Tests

Must cover:

skill normalization
profile completeness
job classification
job freshness
CV deterministic extraction
fit score calculation
recommendation ranking
email idempotency
privacy deletion

8.2 Integration Tests

Must cover:

fixture ingestion -> raw job -> normalized job
normalized job -> skill extraction
CV upload -> parse task -> profile skill
profile update -> stale recommendations
match creation -> MatchResult
recommendation refresh -> JobRecommendation
weekly digest -> EmailEvent

8.3 View Tests

Must cover:

public job search
job detail
quick match
dashboard requires login
CV upload requires login
match requires login
saved jobs require login
admin requires staff

8.4 Manual MVP Flow Tests

Must cover:

new user signs up
uploads CV
completes profile
searches jobs
runs quick match
runs full match
saves job
gets recommendations
deletes CV
deletes account

9. MVP Definition of Done

The MVP is done when:

- User can register/login.
- User can search France IT jobs.
- Jobs come from local database.
- Admin can ingest/manage jobs.
- User can upload CV.
- User can complete profile.
- User can run full match.
- Anonymous user can run quick match.
- User can see recommendations.
- User can save jobs.
- User can manage email preferences.
- User can delete CV/account.
- Admin can inspect failures.
- System can be deployed.

10. Next Planning Document

After this roadmap, the next document is:

MVP Backlog v1

It should convert phases into implementation tickets with:

- ticket title
- type
- description
- dependencies
- acceptance criteria
- test expectations
- priority
- phase

After MVP Backlog v1:

Repository Setup Plan v1

Then:

First Coding Phase